RAG Model for NCERT Science (Class 6–12)

Understanding Chunking for Retrieval-Augmented Generation

Project Mentor : Dr. Ashish Kumar Sahu

Members :
Sarthak Mittal
Chandan Gupta
Aryan Chilkewar
Agrim Salhotra

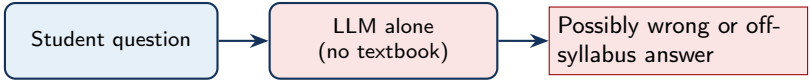
Topics Covered in This Lecture

- What Retrieval-Augmented Generation (RAG) is and why it is used.
- What chunking is, and why the way we cut a document matters.
- Six common chunking strategies, with diagrams, code, advantages and disadvantages.
- How to choose a strategy for a textbook such as NCERT Science.
- Research findings on chunking, explained in simple terms.

Why Not Just Ask the LLM Directly?

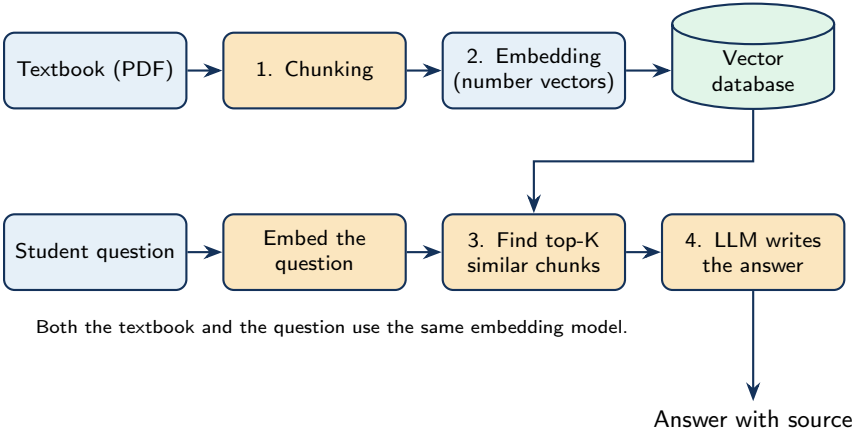
A Large Language Model (LLM) answers from what it learned during training. For school or university content, this causes two problems.

- **Hallucination:** the model can give a confident answer that is wrong.
- **Off-syllabus answers:** the model may use advanced explanations that do not match the textbook.



Retrieval-Augmented Generation (RAG)

RAG first *finds* the relevant text in a document collection. Then it gives that text to the LLM, which answers from it.

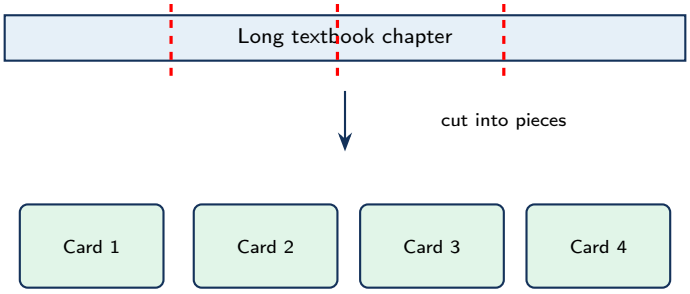


Key Words in Plain Language

- **Token:** a small piece of text that a model reads, often a word or part of a word.
- **Chunk:** one piece of a document that we store and search as a single unit.
- **Embedding:** a list of numbers that represents the meaning of a piece of text.
- **Vector:** the same as an embedding. It is a point in a space with many dimensions.
- **Similarity:** how close two vectors are. Close vectors mean similar meaning.
- **Retrieval:** finding the chunks most similar to the question.
- **Top-K:** the K best-matching chunks. For example, top-5 means the five best.

An Analogy: Library Cards

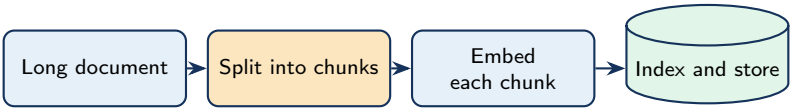
Think of a textbook as a long scroll. Chunking cuts the scroll into cards. Each card gets a label, so a search can find the right card quickly.



Small, labelled cards are easier to search than one very long scroll.

What Is Chunking?

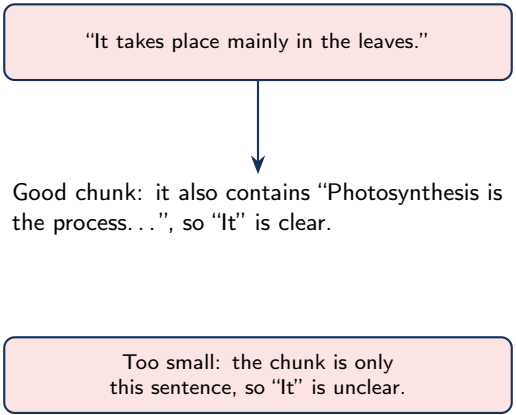
Chunking means splitting a long document into smaller pieces before we create embeddings and store them.



- A chunk that is too large may mix several topics, so its embedding becomes vague.
- A chunk that is too small may lose the context needed to understand it.

Why Chunk Size Matters

The same sentence can mean different things depending on what surrounds it.



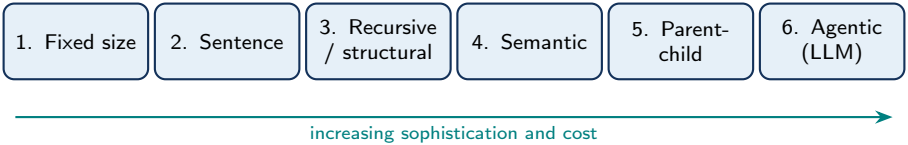
Running Example Used in This Lecture

This short paragraph is used on every strategy slide, so the methods can be compared on the same text.

> Photosynthesis is the process by which green plants prepare food. It takes place mainly in the leaves. Chlorophyll in the chloroplasts absorbs sunlight. Carbon dioxide enters through stomata, and water is absorbed by the roots. The overall equation is $6\mathrm{CO}_2 + 6\mathrm{H}_2\mathrm{O} \rightarrow \mathrm{C}_6\mathrm{H}_{12}\mathrm{O}_6 + 6\mathrm{O}_2$.

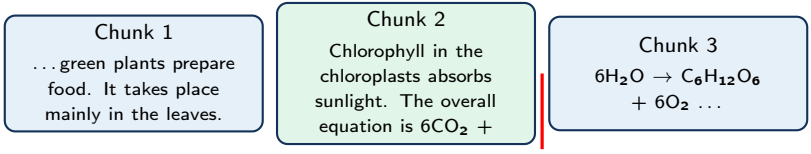
Six Chunking Strategies at a Glance

Strategies are listed from simplest to most complex. More complex methods usually cost more to build and run.



Strategy 1: Fixed-Size Chunking (Idea)

Cut the text every N characters or tokens. An optional overlap repeats a few characters at the start of the next chunk.



Strategy 1: Fixed-Size Chunking (Code and Trade-offs)

```
def fixed_chunks(text, size=60, overlap=0):
    chunks = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start + size])
        start += size - overlap
    return chunks
```

Advantages

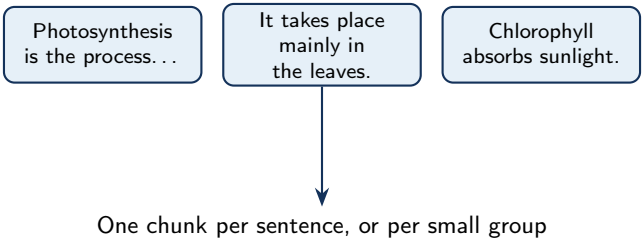
- Very simple to write and fast to run.
- Every chunk has about the same size, which makes storage easy.

Disadvantages

- Ignores meaning, so it can cut sentences, definitions and equations.
- The size must be tuned, and the best value depends on the document.

Strategy 2: Sentence-Based Chunking (Idea)

Split the text at sentence endings, then group one or a few whole sentences into each chunk. No sentence is cut in half.



Strategy 2: Sentence-Based Chunking (Code and Trade-offs)

```
import re

def sentence_chunks(text):
    parts = re.split(r'(?<=[.!?])\s+', text)
    return [p.strip() for p in parts if p.strip()]
```

Advantages

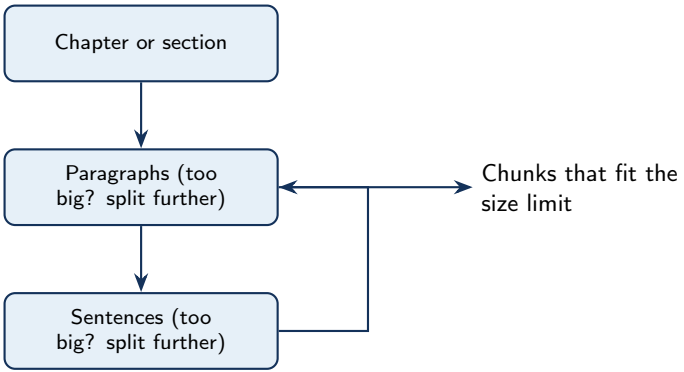
- Never cuts a sentence, so each chunk reads naturally.
- Cheap to compute, and no model is needed.

Disadvantages

- One sentence can lack context. "It takes place..." does not say what "It" is.
- Simple rules fail on abbreviations such as "e.g." and on equations that contain full stops.

Strategy 3: Recursive / Structural Chunking (Idea)

Split by the largest structure first (sections), then by smaller units (paragraphs, then sentences). Keep going until every chunk fits the size limit.



Strategy 3: Recursive / Structural Chunking (Code and Trade-offs)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50,
    separators=["\n\n", "\n", ". ", " ", ""])
chunks = splitter.split_text(chapter_text)
```

Advantages

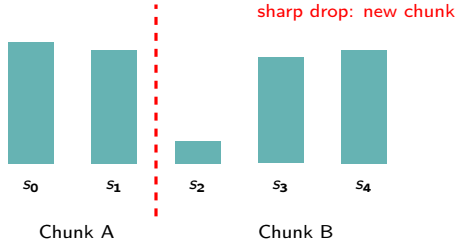
- Follows the document's own structure, so a whole topic can stay together.
- Works well for textbooks that have clear headings.

Disadvantages

- Chunk sizes vary, and very long sections still need splitting.
- Depends on clean text. Poorly extracted headings lead to poor chunks.

Strategy 4: Semantic Chunking (Idea)

Measure how similar each sentence is to the next one. Start a new chunk where similarity drops sharply, because the topic has changed.



Bar height shows how similar a sentence is to the one before it (illustrative values).

Strategy 4: Semantic Chunking (Code and Trade-offs)

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode(sentences, normalize_embeddings=True)

chunks, current = [], [sentences[0]]
for i in range(1, len(sentences)):
    sim = float(np.dot(emb[i-1], emb[i]))
    if sim < 0.5:                # threshold: a tuning choice
        chunks.append(" ".join(current))
        current = []
    current.append(sentences[i])
chunks.append(" ".join(current))
```

Advantages

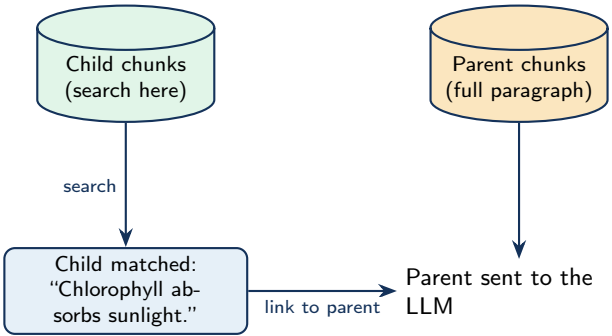
- Follows real topic changes, such as a move from plant cells to animal cells.

Disadvantages

- Needs an embedding for every sentence, which costs time during ingestion.
- The threshold changes the result, so it must be tested on your own data.

Strategy 5: Parent-Child Chunking (Idea)

Search with small *child* chunks, which are precise. Send the larger *parent* chunk to the LLM, which gives more context.



Strategy 5: Parent-Child Chunking (Code and Trade-offs)

```
parents = {"P1": "Photosynthesis is the process ... 6O2."}
children = [
    {"text": "Chlorophyll absorbs sunlight.", "parent": "P1"},
    {"text": "The overall equation is 6CO2 + 6H2O -> C6H12O6 + 6O2.", "parent": "P1"
        },
]

def answer_context(query):
    best = search_children(query)   # vector search over child texts
    return parents[best["parent"]]  # send the whole parent to the LLM
```

Advantages

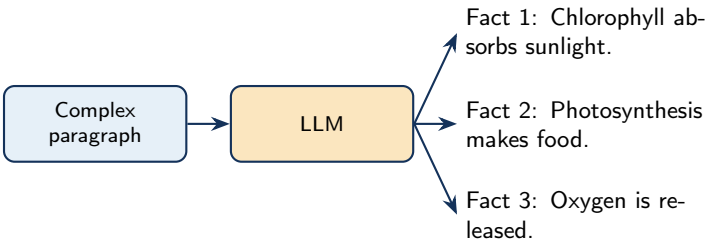
- Precise search and enough context for the answer.
- Useful when a formula is short but its explanation is in the paragraph around it.

Disadvantages

- Needs two stores: a vector index and a document store.
- More code to write and maintain.

Strategy 6: Agentic / LLM-Assisted (Propositional) Chunking

An LLM reads each paragraph and rewrites it as short, standalone facts. Each fact becomes one chunk.



Advantages

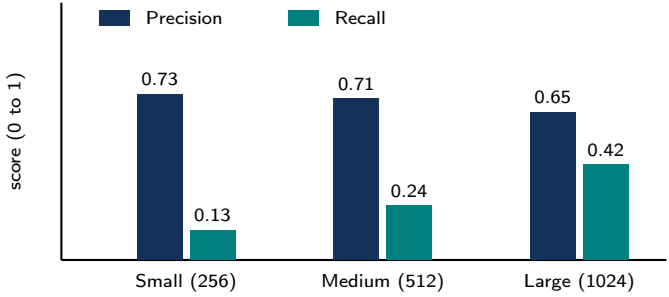
- Each chunk holds one clear fact, which suits short factual questions.

Disadvantages

- Slow and costly, because an LLM must process the whole book before indexing.
- The LLM may change the meaning, so errors are stored in the index.

Trade-off: Small Chunks vs Large Chunks

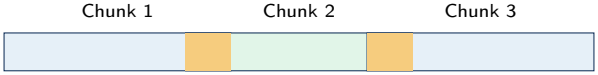
Smaller chunks tend to be more precise. Larger chunks tend to cover more of the answer.



Precision: how many retrieved chunks are relevant. Recall: how much of the relevant material is retrieved.

Overlap: Repeating Some Text Between Chunks

Overlap copies the last few characters or sentences of one chunk to the start of the next, so a boundary does not lose context.

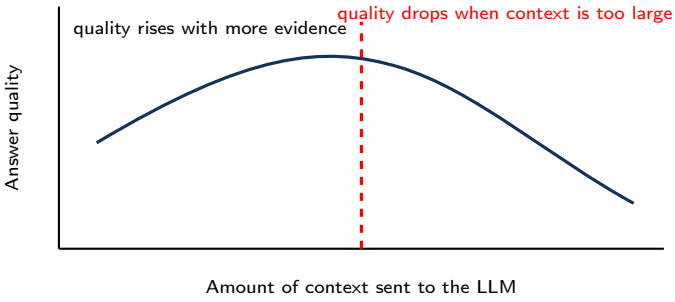


Shaded = repeated text (extra storage)

In a sentence-based study, 10 to 20 percent overlap gave no measurable gain, while the number of chunks and the index size grew. A common default is no overlap, unless tests show it helps.

How Many Chunks to Send to the LLM?

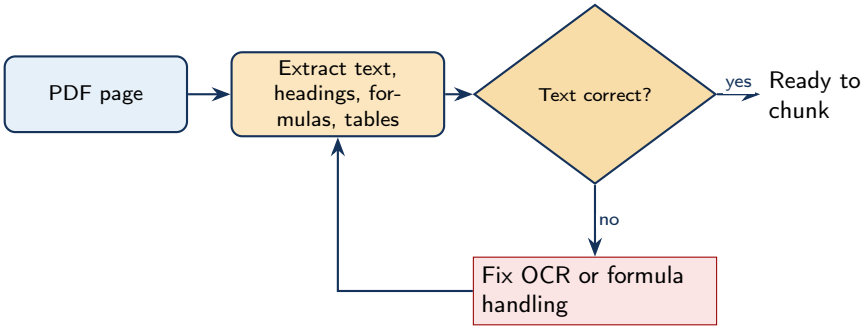
The retrieved chunks fill a context budget. More context is not always better.



The shape is a general pattern. The exact point where quality drops depends on the model, so test it on your own data.

Before Chunking: Extract the Text Correctly

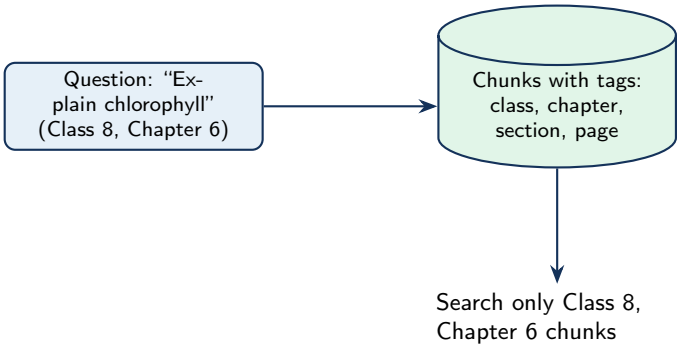
Chunking can only be as good as the text it receives. Extraction errors from PDFs often matter more than the chunk size.



Example: if a chemical equation is extracted as “2H2 O2” with the arrow missing, no chunking method can recover its meaning.

Attach Metadata to Each Chunk

Metadata is extra information stored with each chunk, such as class, chapter, section and page. The search can filter by these tags before it matches meaning.

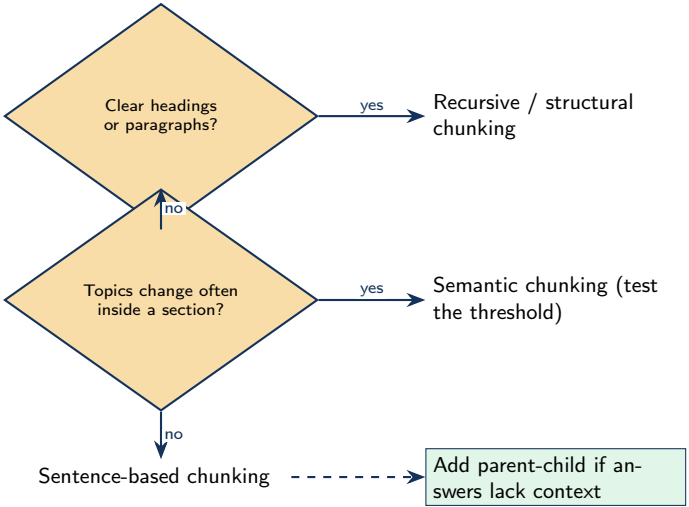


Research Findings in Simple Terms

- Chunking is a major design choice. It changes which text is found and how good the final answer is.
- Splitting that ignores structure, such as fixed character cuts, usually performs worst.
- Methods that keep natural units together, such as paragraphs or sections, often perform better.
- Smaller chunks give higher precision. Larger chunks give higher recall.
- Overlap is not needed in most sentence-based setups, and it adds storage.
- Adding more context does not always help. Quality can fall when too much text is sent.
- A better embedding model and a better chunking method both help. Neither replaces the other.
- No single method is best for every subject. The right choice depends on the text.

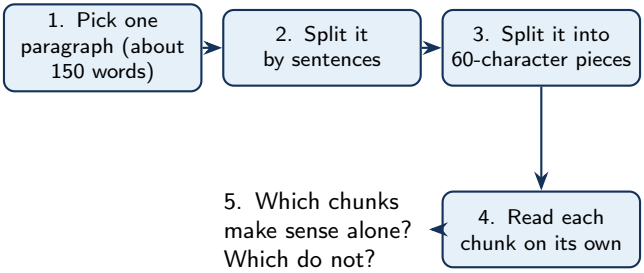
Choosing a Strategy

Start with the text, and follow the questions in the flowchart.



Hands-On Exercise

Try this with one paragraph from any textbook.



Summary

- RAG finds relevant text first, then the LLM answers from that text.
- Chunking decides what a search can find. Good chunks keep meaning together.
- Six strategies exist: fixed size, sentence, recursive, semantic, parent-child and agentic.
- Start with structure-aware chunking. Test the chunk size, and avoid overlap unless tests show a benefit.
- Extract text carefully, and attach metadata such as class, chapter and page.

References

1. M. Bennani and C. Moslonka, "A Systematic Analysis of Chunking Strategies for Reliable Question Answering," arXiv:2601.14123, 2026.
2. M. A. Shaukat, M. Adnan, and C. C. N. Kuhn, "A Systematic Investigation of Document Chunking Strategies and Embedding Sensitivity," arXiv:2603.06976, 2026.
3. D. Yu. Yuvzhenko and S. H. Stirenko, "A Comparative Study of Chunking Strategies for Retrieval-Augmented Generation," *Cybernetics and Systems Analysis*, vol. 62, no. 3, 2026. DOI: 10.1007/s10559-026-00874-3.
4. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv:2005.11401, 2020.
5. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, 2019.
6. LangChain documentation, Text Splitters (RecursiveCharacterTextSplitter).